C N 2025-10-28

# Metodo di Newton modificato.

```
function [x,it,xx] = newton(fun,x0,tol,...
                            maxit,molt)
%
% [x,it,xx] = newton(fun,x0,tol,maxit,molt)
%
% Metodo di Newton per gli zeri di una funzione.
%
% Input:
%
%    fun - identificatore function che implementa
%          il problema da risolvere, del tipo
%
%                [fx,f1x] = fun(x),
%
%          con fx e f1x, rispettivamente i
%          valori della funzione e della sua
%          derivata prima, calcolati in x;
%     x0 - approssimazione iniziale;
%    tol - tolleranza per il criterio di arresto
%          (default = 1E-8);
%  maxit - numero massimo di iterazioni
%          (default = 1000);
%   molt - molteplicità della radice
%          (default = 1).
%
% Output:
%
%      x - approssimazione trovata;
%     it - numero di iterazioni per ottenere
%          convergenza. Se it=-1, allora
%          la convergenza non è stata
%          ottenuta entro le iterazioni massime
```

```
%                (opzionale);
%        xx - vettore con le approssimazioni ottenute
%                (opzionale).
%
%                                        Rel. 2025-10-28.
%

if nargin < 2
    error('numero argomenti insufficiente');
end
if nargin >= 3
    if tol <= 0
        error('tolleranza non idonea');
    end
    if nargin >= 4
        if maxit < 1
            error('numero massimo iterazioni errato');
        end
        if nargin >= 5
            if molt < 1 || molt ~= fix(molt)
                error('molteplicità non corretta');
            end
        else
            molt = 1
        end
    else
        maxit = 1000;
        molt = 1;
    end
else
    tol = 1e-8
    maxit = 1000;
    molt = 1;
end
```

```
if nargout == 3
    flag = 1;
    xx = zeros(maxit+1, 1);
    xx(1) = x0;
else
    flag = 0;
end

for i = 1:maxit
    [fx, f1x] = feval(fun, x0);
    if f1x == 0 && fx == 0
        x = x0;
        if flag, xx = xx(1:i), it=i; end
    elseif f1x == 0
        error('metodo non applicabile');
        it = -1;
        if flag, xx = xx(1:i), end
    else
        x = x0 - molt * fx / f1x;
        if flag, xx(i+1) = x; end
    end
    err = abs(x - x0) / (1 + abs(x0));
```

```
        if  err <= tol,
            it=i;
            if flag, xx = xx(1:i); end
            break
        end
    xo = x;
end
if err > tol, it=-1; end
return
```

## Risoluzione di sistemi lineari.

N.B. : riguardare l'Appendice A1.

Il problema è risolvere un sistema di equazioni lineari, della forma:

$$(1)\quad \begin{cases} a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n = b_1, \\ a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n = b_2, \\ \vdots \\ a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n = b_m, \end{cases}$$

in cui i coefficienti $a_{ij}$, $i=1,\dots,m$, $j=1,\dots,n$ sono assegnati, come lo sono anche i termini noti $b_1,\dots,b_m$. Le incognite da determinare sono $x_1,\dots,x_n$.

È possibile scrivere il sistema di equazioni (1), in forma vettoriale, introducendo la **matrice dei coefficienti**

$$A=\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} \in \mathbb{R}^{m\times n},$$

il **vettore dei termini noti**,

$$\underline{b}=\begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix} \in \mathbb{R}^{m},$$

e il **vettore delle incognite**,

$$\underline{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^n ,$$

come:

$$A \cdot \underline{x} = \underline{b} . \qquad (2)$$

Nella nostra trattazione, assumeremo che $m \geq n$, ovvero che il numero delle equazioni non sia minore del numero delle incognite. Pertanto il numero di colonne della matrice $A$ è $\leq$ numero di righe.

**Memo**

- $(a_{i1}, \dots, a_{in}) \in \mathbb{R}^{1 \times n}$, è la $i$-esima riga di $A$;
- $\begin{bmatrix} a_{1j} \\ a_{2j} \\ \vdots \\ a_{mj} \end{bmatrix} \in \mathbb{R}^m$, è la $j$-esima colonna di $A$;
- $a_{ij}$ è l'elemento che si trova nella intersezione della riga $i$-esima con la colonna $j$-esima.

Assumeremo, inoltre, che la matrice $A$ abbia rango massimo. Pertanto uguale ad $n$. Questo significa che le colonne di $A$ sono vettori linearmente indipendenti tra loro.

Distinguiamo 2 casi significativi che sono il caso in cui

1) $m = n \iff A$ è una **matrice quadrata**;

2) $m > n \iff A$ è a rango massimo.

## Il caso quadrato

Se $A \in \mathbb{R}^{n \times n}$, e $rank(A) = n$, segue che $A$ è una matrice **nonsingolare**.

Questo significa che $\exists\, A^{-1}$, la matrice inversa di $A$, tale che

$$A^{-1} \cdot A = A \cdot A^{-1} = I = \begin{bmatrix} 1 & & \\ & \ddots & \\ & & 1 \end{bmatrix} \in \mathbb{R}^{n \times n},$$

con $I$ la matrice identità.

OSS In questo caso $\underline{x}, \underline{b} \in \mathbb{R}^n$ e, evidentemente $I \cdot \underline{x} = \underline{x}$.

Pertanto, se dobbiamo risolvere

$$A\underline{x} = \underline{b}$$

allora, moltiplicando membro a membro per $A^{-1}$, otteniamo che $\exists!$

$$\underline{x} = A^{-1}\underline{b},$$ soluzione del problema. Tuttavia, questa espressione della soluzione non è generalmente efficiente dal punto di vista computazionale. Pertanto, sarà utilizzata solo in casi molto particolari.

Ricordiamo che

$$A \text{ non singolare} \iff \det(A) \neq 0.$$

Cosa che assumeremo nel seguito.

Cominciamo con l'esame di casi di

sistemi lineari "semplici". Questi casi sono quelli in cui la matrice A è:

- diagonale
- triangolare
- ortogonale

La loro elaborazione è fatta per complessità computazionale crescente. Quest'ultima è misurata in termini di occupazione di memoria e numero di operazioni algebriche richieste per la risoluzione del sistema.

A diagonale: in questo caso $a_{ij} = 0,\ i \neq j$.

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \cdots & a_{2n} \\ a_{31} & a_{32} & a_{33} & \cdots & a_{3n} \\ \vdots & & \vdots & & \vdots \\ a_{n1} & a_{n2} & a_{n3} & \cdots & a_{nn} \end{bmatrix}$$

**OSS** Se $a_{ij}$ è il generico elemento in riga $i$ e colonna $j$ di $A$, allora la differenza $j-i$:

- $= 0$, per gli elementi sulla **diagonale principale**;
- $= k > 0$, per gli elementi sulla **$k$-esima sopra diagonale**;
- $= r < 0$, per gli elementi sulla **$(-k)$-esima sottodiagonale**.

Nel caso in cui $A$ è diagonale, si ottiene:

$$A = \begin{bmatrix} a_{11} & & & \\ & a_{22} & & \\ & & \ddots & \\ & & & a_{nn} \end{bmatrix}$$

$\Rightarrow$ è sufficiente un vettore di lunghezza $n$ per memorizzare gli elementi significativi di $A$.

$A$ è un caso particolare di **matrice sparsa**, ovvero una matrice il cui elementi nonnulli sono $O(n)$, invece che $n^2$.

In questo caso

$$A\underline{x} = \underline{b}$$

diviene, semplicemente,

$$a_{11} x_1 = b_1$$
$$a_{22} x_2 = b_2$$
$$\vdots$$
$$a_{nn} x_n = b_n$$

e, considerato che $\det(A) = \prod_{i=1}^{n} a_{ii} \neq 0$, segue che $a_{ii} \neq 0$, $\forall i = 1, \dots, n$. Pertanto la soluzione si ottiene con:

$$x_i = b_i / a_{ii}, \quad i = 1, \dots, n.$$

In conclusione, sono sufficienti 2 vettori di lunghezza $n$ (uno per la diagonale di $A$ e l'altro per il termine noto, che possiamo riscrivere con $\underline{x}$) e $n$ operazioni algebriche elementari.